

Contact Management API — Adding PostgreSQL Persistence

Extended the Contact Management API to store data in a real database instead of an in-memory list, replacing the storage layer without changing the API's public behavior.

1. Database Setup

Ran PostgreSQL in a Docker container:

```
docker run -d --name postgresdb \
  -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 -v contactsdb_pgdata:/var/lib/postgresql/data postgres:16
```

Isolated to its own container, volume, and port — independent of any other project on the machine.

2. Schema Creation

Created the database and table via SQL:

```
CREATE DATABASE contacts_db;

CREATE TABLE contacts (
  id      SERIAL PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  email   VARCHAR(150),
  phone   VARCHAR(50)
);
```

Column types mirror the `Contact` model: `id` auto-increments to match the API's identifier field, `name` is required, `email` and `phone` are optional.

3. Data Access Package

Added `Npgsql`, the PostgreSQL driver for .NET, to the project via NuGet.

4. Connection Configuration

Stored the connection string in `appsettings.json` rather than in code, so it can change per environment without a code change:

```
"ConnectionStrings": {
  "DefaultConnection": "Host=localhost;Port=5432;Username=postgres;Password=postgres;Database=contacts_db"
}
```

5. PostgreSQL Repository

Implemented `PostgresContactRepository`, a second implementation of the existing `IContactRepository` interface. Each method opens a connection, runs one parameterized SQL statement, and maps the result back to a `Contact` object:

Operation	SQL
GetAll	SELECT ... FROM contacts ORDER BY id
GetById	SELECT ... WHERE id = @id
Create	INSERT ... RETURNING id
Update	UPDATE ... WHERE id = @id
Delete	DELETE ... WHERE id = @id

All values are passed as parameters (@id, @name, etc.) rather than concatenated into the query string, preventing SQL injection.

6. Dependency Injection

Swapped the repository registration in the application's service container from the in-memory implementation to the PostgreSQL one:

```
builder.Services.AddScoped<IContactRepository, PostgresContactRepository>();
```

No other file changed. The controller depends only on the `IContactRepository` interface, so it had no knowledge of the underlying storage mechanism to begin with.

7. Verification

Confirmed persistence end to end: created a contact through the API, queried the database directly to confirm the row existed independently of the API's response, updated and deleted it through the API, and re-queried the database to confirm each change was reflected — including restarting the application and confirming previously created data was still present, proving storage was no longer tied to the process lifetime.

Checklist against the exercise

- ☑ Set up and start PostgreSQL via Docker — command documented in Step 1
- ☑ Create the database and table using SQL statements — Step 2
- ☑ Replace the in-memory collection with ADO.Net-based CRUD operations against the database — Steps 3–6